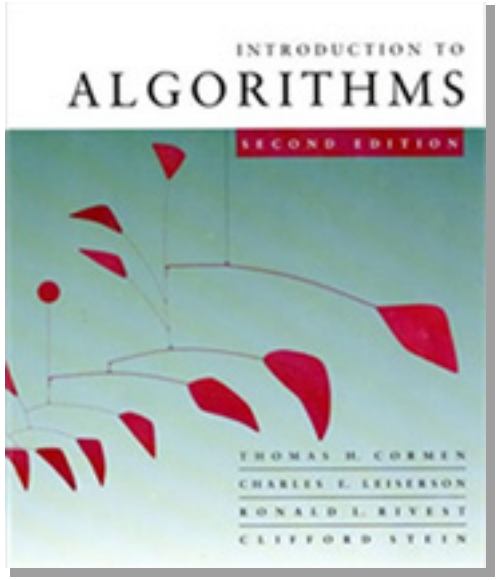
# *Introduction to Algorithms*

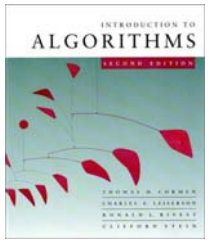



## LECTURE 7

**Hashing I**

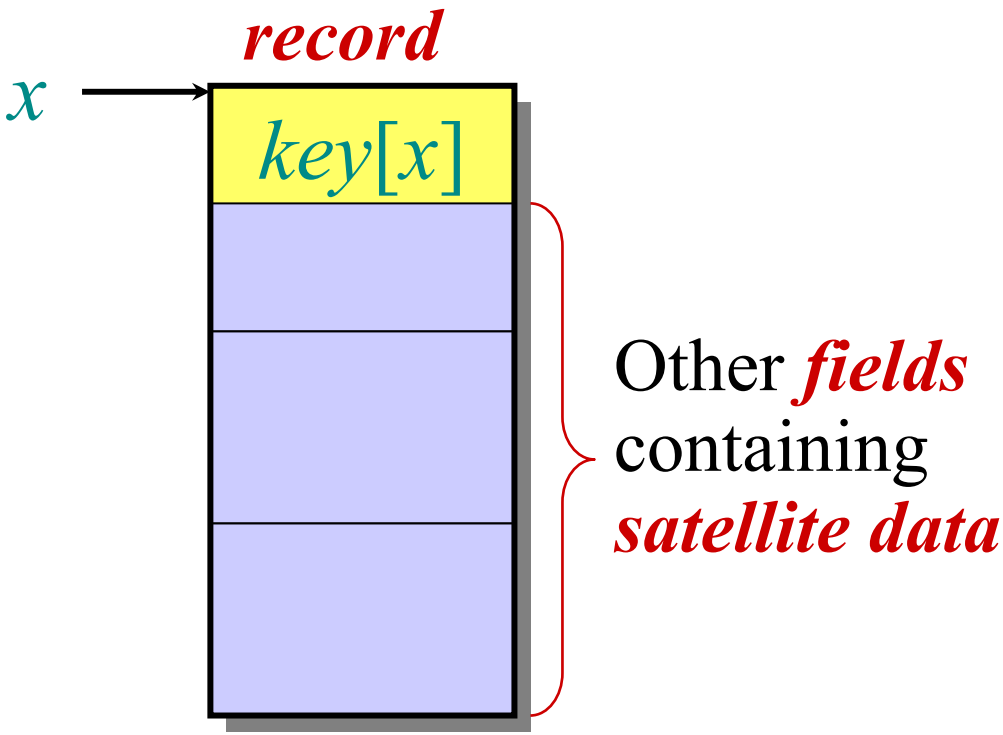
- Direct-access tables
- Resolving collisions by chaining
- Choosing hash functions
- Open addressing


*Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson*




# Symbol-table problem

Symbol table $S$ holding $n$ ***records***:

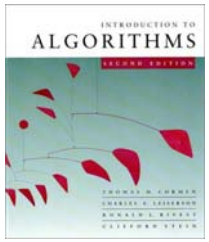



Operations on $S$:

- INSERT$(S, x)$
- DELETE$(S, x)$
- SEARCH$(S, k)$

How should the data structure $S$ be organized?


*Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson*




# Direct-access table

**IDEA:** Suppose that the keys are drawn from the set $U \subseteq \{0, 1, \ldots, m-1\}$, and keys are distinct. Set up an array $T[0 \,.\,.\, m-1]$:

$$T[k] = \begin{cases} x & \text{if } x \in S \text{ and } key[x] = k, \\ \text{NIL} & \text{otherwise.} \end{cases}$$

Then, operations take $\Theta(1)$ time.

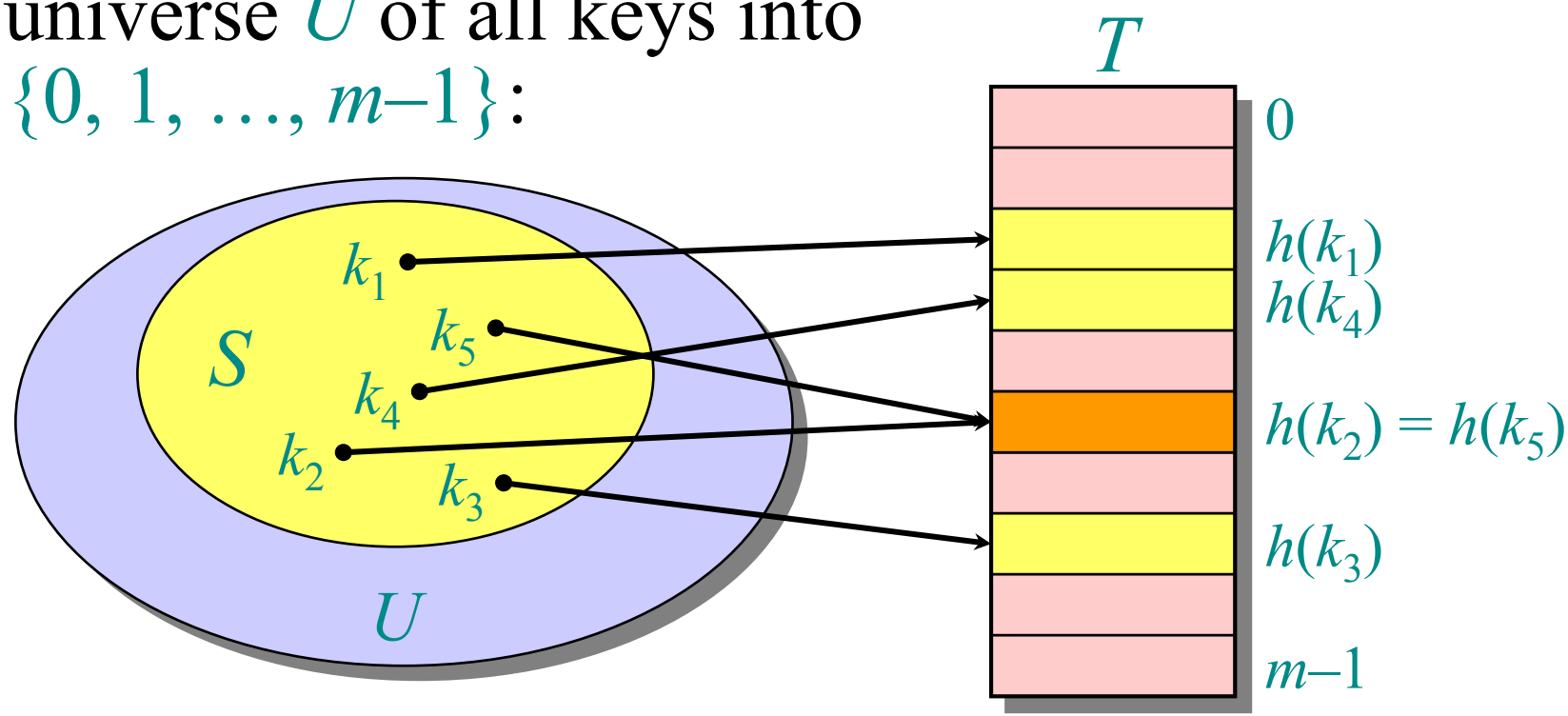
**Problem:** The range of keys can be large:

- 64-bit numbers (which represent 18,446,744,073,709,551,616 different keys),
- character strings (even larger!).

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 

# Hash functions

**Solution:** Use a ***hash function*** $h$ to map the universe $U$ of all keys into $\{0, 1, \ldots, m-1\}$:

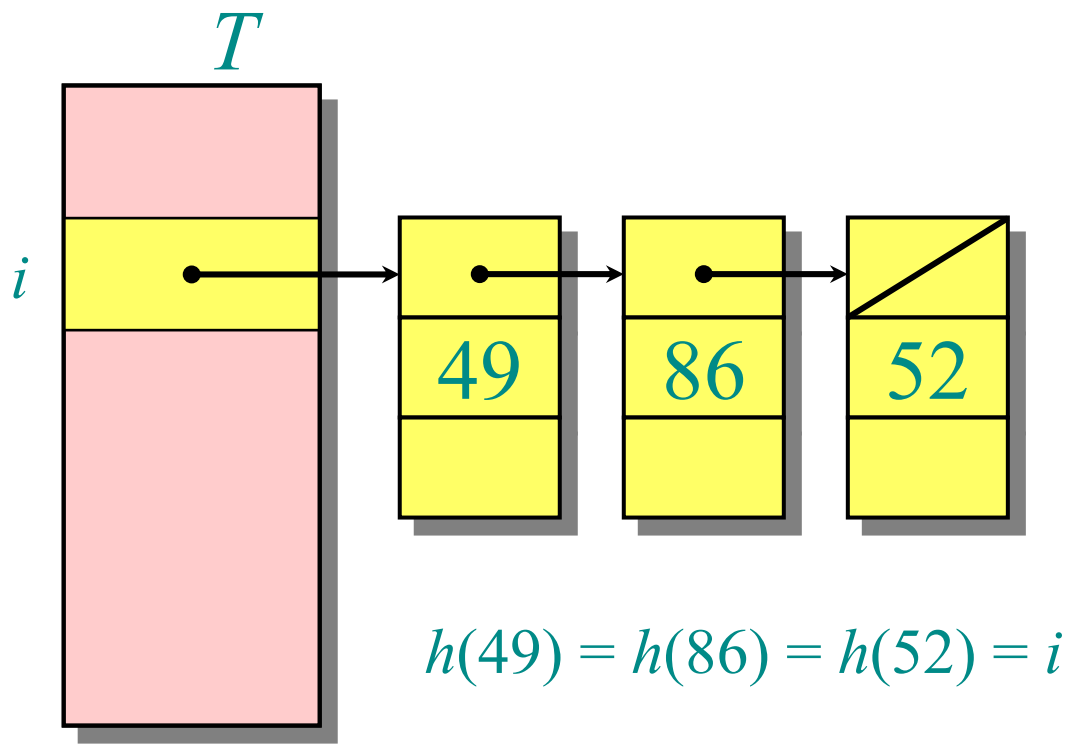



When a record to be inserted maps to an already occupied slot in $T$, a ***collision*** occurs.


*Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson*

# Resolving collisions by chaining

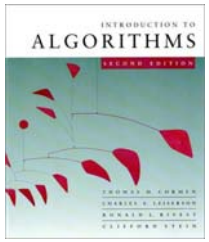
- Link records in the same slot into a list.




***Worst case:***

- Every key hashes to the same slot.
- Access time = $\Theta(n)$ if $|S| = n$


*Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson*




# Average-case analysis of chaining

We make the assumption of ***simple uniform hashing:***

- Each key $k \in S$ is equally likely to be hashed to any slot of table $T$, independent of where other keys are hashed.

Let $n$ be the number of keys in the table, and let $m$ be the number of slots.

Define the ***load factor*** of $T$ to be

$$\alpha = n/m$$

$$= \text{average number of keys per slot.}$$

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 

# Search cost

The expected time for an ***unsuccessful*** search for a record with a given key is $= \Theta(1 + \alpha)$.

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 

# Search cost

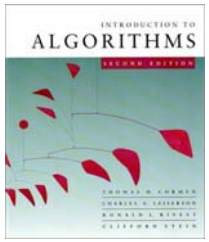
The expected time for an ***unsuccessful*** search for a record with a given key is

$= \Theta(1 + \alpha)$.

*search the list*

*apply hash function and access slot*


*Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson*




# Search cost

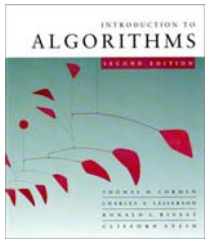
The expected time for an ***unsuccessful*** search for a record with a given key is $= \Theta(1 + \alpha)$.

*search the list*

*apply hash function and access slot*

Expected search time $= \Theta(1)$ if $\alpha = O(1)$, or equivalently, if $n = O(m)$.

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 




# Search cost

The expected time for an ***unsuccessful*** search for a record with a given key is $= \Theta(1 + \alpha)$.

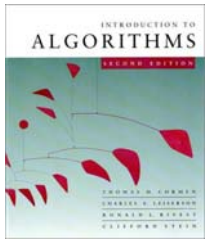
*search the list*

*apply hash function and access slot*

Expected search time $= \Theta(1)$ if $\alpha = O(1)$, or equivalently, if $n = O(m)$.

A ***successful*** search has same asymptotic bound, but a rigorous argument is a little more complicated. (See textbook.)

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 



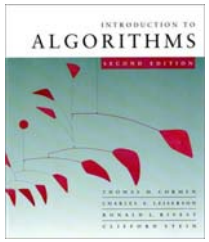

# Choosing a hash function

The assumption of simple uniform hashing is hard to guarantee, but several common techniques tend to work well in practice as long as their deficiencies can be avoided.

**Desirata:**

- A good hash function should distribute the keys uniformly into the slots of the table.
- Regularity in the key distribution should not affect this uniformity.

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 




# Division method

Assume all keys are integers, and define

$$h(k) = k \bmod m.$$

**Deficiency:** Don't pick an $m$ that has a small divisor $d$. A preponderance of keys that are congruent modulo $d$ can adversely affect uniformity.

**Extreme deficiency:** If $m = 2^r$, then the hash doesn't even depend on all the bits of $k$:

- If $k = 1011000111\underbrace{011010}_{h(k)}{}_2$ and $r = 6$, then $h(k) = 011010_2$.

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 

# Division method (continued)

$$h(k) = k \bmod m.$$

Pick $m$ to be a prime not too close to a power of 2 or 10 and not otherwise used prominently in the computing environment.

**Annoyance:**

- Sometimes, making the table size a prime is inconvenient.

But, this method is popular, although the next method we'll see is usually superior.

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 

# Multiplication method

Assume that all keys are integers, $m = 2^r$, and our computer has $w$-bit words. Define

$$h(k) = (A \cdot k \bmod 2^w) \text{ rsh } (w - r),$$

where rsh is the "bitwise right-shift" operator and $A$ is an odd integer in the range $2^{w-1} < A < 2^w$.

- Don't pick $A$ too close to $2^{w-1}$ or $2^w$.
- Multiplication modulo $2^w$ is fast compared to division.
- The rsh operator is fast.

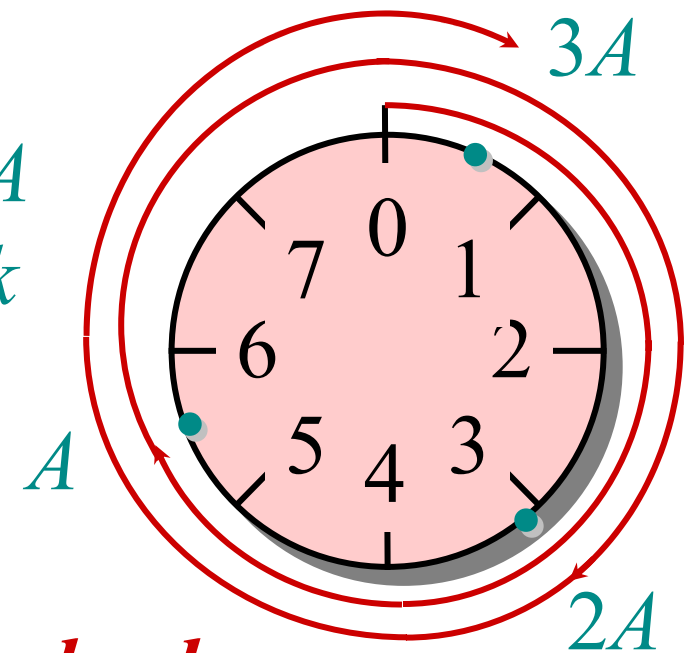

*Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson*

# Multiplication method example

$$h(k) = (A \cdot k \bmod 2^w) \text{ rsh } (w - r)$$

Suppose that $m = 8 = 2^3$ and that our computer has $w = 7$-bit words:

$$\begin{array}{rl} 1\,0\,1\,1\,0\,0\,1 & = A \\ \times \quad 1\,1\,0\,1\,0\,1\,1 & = k \\ \hline 1\,0\,0\,1\,0\,1\,0\,\underbrace{0\,1\,1}_{h(k)}\,0\,0\,1\,1 & \end{array}$$




***Modular wheel***


*Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson*

# Resolving collisions by open addressing

No storage is used outside of the hash table itself.

- Insertion systematically probes the table until an empty slot is found.
- The hash function depends on both the key and probe number:

  $$h : U \times \{0, 1, \ldots, m-1\} \rightarrow \{0, 1, \ldots, m-1\}.$$

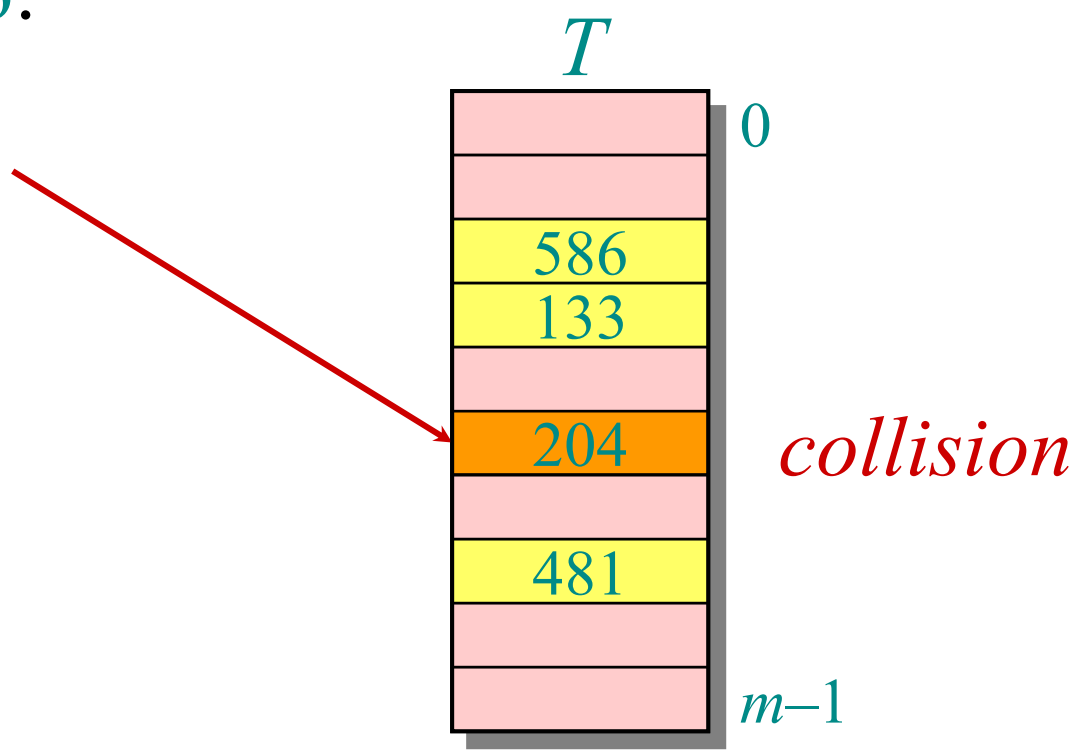
- The probe sequence $\langle h(k,0), h(k,1), \ldots, h(k,m-1)\rangle$ should be a permutation of $\{0, 1, \ldots, m-1\}$.
- The table may fill up, and deletion is difficult (but not impossible).

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 

# Example of open addressing

Insert key $k = 496$:

0. Probe $h(496,0)$

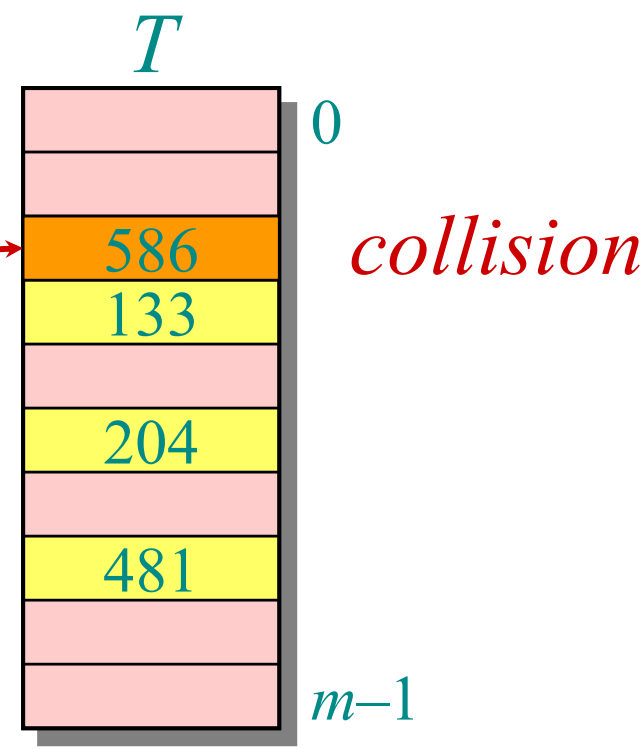



 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 

# Example of open addressing

Insert key $k = 496$:

0. Probe $h(496,0)$
1. Probe $h(496,1)$



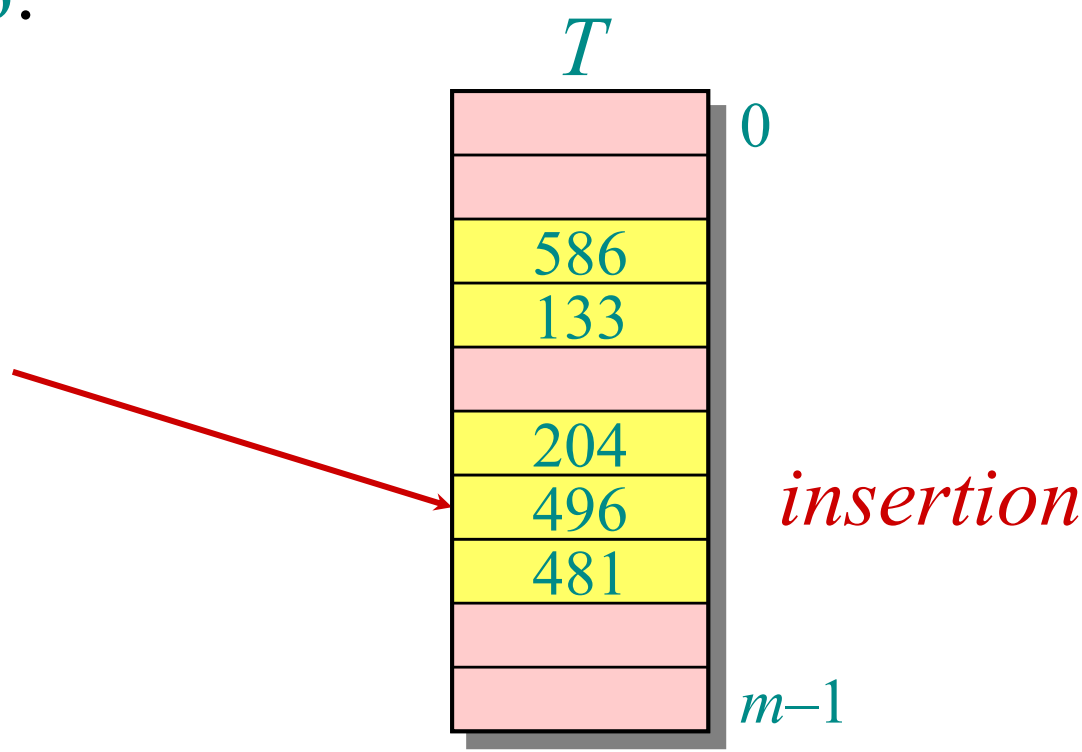


*Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson*

# Example of open addressing

Insert key $k = 496$:

0. Probe $h(496,0)$
1. Probe $h(496,1)$
2. Probe $h(496,2)$



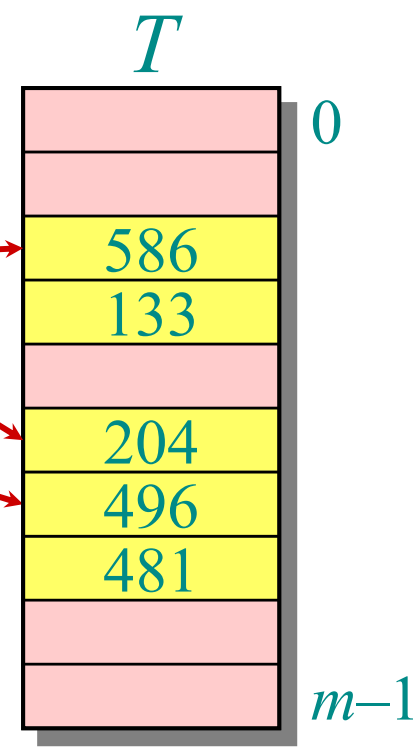


*Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson*

# Example of open addressing

Search for key $k = 496$:

0. Probe $h(496,0)$
1. Probe $h(496,1)$
2. Probe $h(496,2)$




Search uses the same probe sequence, terminating successfully if it finds the key and unsuccessfully if it encounters an empty slot.


*Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson*

# Probing strategies

**Linear probing:**

Given an ordinary hash function $h'(k)$, linear probing uses the hash function

$$h(k,i) = (h'(k) + i) \bmod m.$$

This method, though simple, suffers from ***primary clustering***, where long runs of occupied slots build up, increasing the average search time. Moreover, the long runs of occupied slots tend to get longer.

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 

# Probing strategies

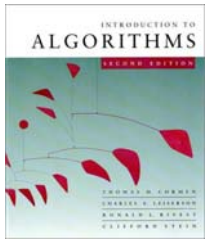
## Double hashing

Given two ordinary hash functions $h_1(k)$ and $h_2(k)$, double hashing uses the hash function

$$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m.$$

This method generally produces excellent results, but $h_2(k)$ must be relatively prime to $m$. One way is to make $m$ a power of $2$ and design $h_2(k)$ to produce only odd numbers.

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 



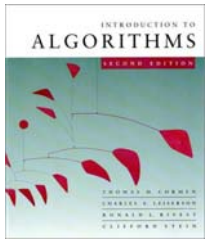

# Analysis of open addressing

We make the assumption of ***uniform hashing:***

- Each key is equally likely to have any one of the $m!$ permutations as its probe sequence.

**Theorem.** Given an open-addressed hash table with load factor $\alpha = n/m < 1$, the expected number of probes in an unsuccessful search is at most $1/(1-\alpha)$.

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 



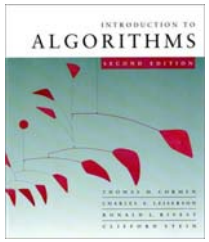

# Proof of the theorem

*Proof.*

- At least one probe is always necessary.
- With probability $n/m$, the first probe hits an occupied slot, and a second probe is necessary.
- With probability $(n-1)/(m-1)$, the second probe hits an occupied slot, and a third probe is necessary.
- With probability $(n-2)/(m-2)$, the third probe hits an occupied slot, etc.

Observe that $\frac{n-i}{m-i} < \frac{n}{m} = \alpha$ for $i = 1, 2, \ldots, n$.

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 




# Proof (continued)

Therefore, the expected number of probes is

$$1+\frac{n}{m}\left(1+\frac{n-1}{m-1}\left(1+\frac{n-2}{m-2}\left(\cdots\left(1+\frac{1}{m-n+1}\right)\cdots\right)\right)\right)$$

$$\le 1+\alpha(1+\alpha(1+\alpha(\cdots(1+\alpha)\cdots)))$$

$$\le 1+\alpha+\alpha^2+\alpha^3+\cdots$$

$$=\sum_{i=0}^{\infty}\alpha^i$$

$$=\frac{1}{1-\alpha}. \square$$

*The textbook has a more rigorous proof and an analysis of successful searches.*

 *Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson* 

# Implications of the theorem

- If $\alpha$ is constant, then accessing an open-addressed hash table takes constant time.
- If the table is half full, then the expected number of probes is $1/(1–0.5) = 2$.
- If the table is 90% full, then the expected number of probes is $1/(1–0.9) = 10$.


*Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson*